

Synthese des Fonctions R

Introduction aux fonctions R

Ce document presente une synthese des principales fonctions R utilisees dans les travaux pratiques de statistiques. Pour chaque fonction, vous trouverez une description de son utilite, sa syntaxe et des exemples d'utilisation.

1. Manipulation des donnees

1.1 Importation et exploration des donnees

read.table(), read.csv()

Fonctions permettant de charger des donnees depuis un fichier. `read.table()` est plus general, tandis que `read.csv()` est specifique aux fichiers CSV.

```
donnees <- read.table("fichier.txt", header = TRUE)
donnees <- read.csv("fichier.csv")
```

View()

Affiche le tableau de donnees dans une interface similaire a Excel. Utile pour explorer visuellement les donnees.

```
View(donnees)
```

nrow(), ncol()

Retournent respectivement le nombre de lignes (observations) et de colonnes (variables) d'un tableau de donnees.

```
nrow(donnees) # nombre d'observations
ncol(donnees) # nombre de variables
```

rownames(), colnames()

Affichent ou modifient les noms des lignes et des colonnes d'un tableau de donnees.

```
colnames(donnees) # noms des variables
colnames(donnees)[3] <- "Nouvelle_Variable" # renommer la 3e colonne
```

str()

Affiche la structure d'un objet R, notamment le type de chaque variable (numerique, facteur, etc.).

```
str(donnees)
```

1.2 Acces aux donnees

donnees[lignes, colonnes]

Permet d'accéder a un sous-ensemble du tableau par position. On peut specifier des indices, des noms ou des conditions logiques.

```
donnees[1:5, 3] # 5 premieres lignes de la 3e colonne
donnees[donnees$age > 30, ] # lignes ou age > 30
```

donnees\$variable

Acces direct a une colonne par son nom via l'operateur \$.

```
ages <- donnees$age # extrait la colonne 'age'
```

donnees[["variable"]]

Acces a une colonne par son nom via la notation [[]].

```
ages <- donnees[["age"]] # equivalent a donnees$age
```

1.3 Manipulation des donnees

names()

Permet d'accéder ou de modifier les noms des colonnes d'un tableau.

```
names(donnees)[names(donnees) == "ancien_nom"] <- "nouveau_nom"
```

Synthese des Fonctions R

ifelse()

Fonction conditionnelle qui retourne une valeur si la condition est vraie, une autre sinon. Utile pour recodage.

```
donnees$categorie <- ifelse(donnees$age < 18, "Mineur", "Majeur")
```

sapply()

Applique une fonction a chaque element d'une liste ou a chaque colonne d'un tableau. Utile pour verifier le type des variables.

```
colonnes_numeriques <- donnees[sapply(donnees, is.numeric)]
```

table()

Cree un tableau de contingence des effectifs pour une ou plusieurs variables categoriques.

```
table(donnees$sexe) # effectifs par sexe
```

```
table(donnees$sexe, donnees$categorie) # tableau croise
```

rm()

Supprime des objets de l'environnement R.

```
rm(variable_temp) # supprime une variable
```

```
rm(list = ls()) # supprime tous les objets
```

2. Echantillonnage et statistique descriptive

2.1 Echantillonnage

sample()

Permet de tirer aleatoirement des elements d'un vecteur. Utile pour creer des echantillons.

```
indices <- sample(nrow(donnees), 50) # 50 indices aleatoires
echantillon <- donnees[indices, ] # echantillon de 50 observations
```

2.2 Statistiques descriptives

length()

Retourne la taille (nombre d'elements) d'un vecteur.

```
n <- length(donnees$age) # nombre d'observations pour la variable age
```

mean()

Calcule la moyenne arithmetique d'un vecteur numerique.

```
moyenne_age <- mean(donnees$age) # moyenne des ages
moyenne_age <- mean(donnees$age, na.rm = TRUE) # ignore les valeurs manquantes
```

sd()

Calcule l'ecart-type d'un vecteur numerique.

```
ecart_type <- sd(donnees$age) # ecart-type des ages
```

median()

Calcule la mediane d'un vecteur numerique (valeur qui separe la distribution en deux parties egales).

```
mediane <- median(donnees$age) # mediane des ages
```

data.frame()

Cree un tableau de donnees a partir de vecteurs.

```
resultats <- data.frame(
  Variable = c("Age", "Taille"),
  Moyenne = c(mean(age), mean(taille)),
  Ecart_Type = c(sd(age), sd(taille))
)
```

2.3 Loi normale

pnorm()

Calcule la fonction de repartition de la loi normale : $P(X \leq x)$.

```
pnorm(1.96) # P(Z <= 1.96) pour une loi N(0,1)
pnorm(100, mean = 90, sd = 5) # P(X <= 100) pour N(90,5)
```

qnorm()

Calcule le quantile d'ordre p de la loi normale.

```
qnorm(0.975) # quantile d'ordre 97.5% de N(0,1)
qnorm(0.95, mean = 100, sd = 15) # quantile d'ordre 95% de N(100,15)
```

3. Graphiques en R

3.1 Histogrammes

hist()

Cree un histogramme pour visualiser la distribution d'une variable numerique.

```
hist(donnees$age, col = "skyblue", main = "Distribution des ages")
hist(donnees$age, breaks = 10, probability = TRUE) # en densite avec 10 classes
```

density()

Estime la densite de probabilite d'une variable. Souvent utilisee avec hist() et lines().

```
hist(donnees$age, probability = TRUE)
lines(density(donnees$age), col = "red", lwd = 2)
```

3.2 Diagrammes en batons

barplot()

Cree un diagramme en batons, utile pour les variables categoriques.

```
barplot(table(donnees$categorie), col = "lightgreen")
barplot(table(donnees$sexe, donnees$categorie), beside = TRUE, legend = TRUE) # groupes
```

3.3 Nuages de points

plot()

Fonction graphique polyvalente, souvent utilisee pour les nuages de points.

```
plot(donnees$taille, donnees$poids, xlab = "Taille", ylab = "Poids")
plot(donnees$taille, donnees$poids, pch = 19, col = "blue") # personnalisation
```

points()

Ajoute des points a un graphique existant. Utile pour superposer plusieurs series.

```
plot(x1, y1, col = "red")
points(x2, y2, col = "blue", pch = 17)
```

legend()

Ajoute une legende a un graphique.

```
legend("topleft", legend = c("Groupe A", "Groupe B"), col = c("red", "blue"), pch = c(1, 17))
```

3.4 Boites a moustaches

boxplot()

Cree une boite a moustaches, utile pour comparer des distributions.

```
boxplot(donnees$age, col = "lightblue")
boxplot(age ~ sexe, data = donnees, col = c("pink", "lightblue")) # par groupe
```

4. Intervalles de confiance

4.1 Loi normale et Student

qnorm()

Utilise pour obtenir les quantiles de la loi normale, nécessaires pour les IC avec grands échantillons.

```
z <- qnorm(0.975) # pour un IC a 95% (bilateral)
```

qt()

Utilise pour obtenir les quantiles de la loi de Student, nécessaires pour les IC avec petits échantillons.

```
t <- qt(0.975, df = n-1) # pour un IC a 95% avec n-1 degres de liberte
```

4.2 Calcul d'intervalles de confiance

Intervalle de confiance pour une moyenne

Formule pour calculer un IC a $(1-\alpha)\%$ pour une moyenne.

```
# Grand échantillon (loi normale)
```

```
moyenne + c(-1,1) * qnorm(1-alpha/2) * ecart_type/sqrt(n)
```

```
# Petit échantillon (loi de Student)
```

```
moyenne + c(-1,1) * qt(1-alpha/2, df=n-1) * ecart_type/sqrt(n)
```

sprintf()

Fonction de formatage de texte, utile pour présenter les résultats des IC.

```
sprintf("IC 95% : [%.2f ; %.2f]", borne_inf, borne_sup)
```

5. Tests statistiques

5.1 Tests de conformité

t.test()

Realise un test de Student pour comparer une moyenne a une valeur de reference ou comparer deux moyennes.

```
# Test de conformité
```

```
t.test(echantillon, mu = valeur_reference)
```

```
# Comparaison de deux échantillons
```

```
t.test(echantillon1, echantillon2)
```

var.test()

Realise un test de Fisher pour comparer deux variances.

```
var.test(echantillon1, echantillon2)
```

Synthese des Fonctions R

5.2 Tests d'homogeneite

t.test() pour echantillons independants

Compare les moyennes de deux groupes independants.

```
t.test(valeur ~ groupe, data = donnees)
t.test(groupe1, groupe2, var.equal = TRUE) # si variances homogenes
```

t.test() pour echantillons apparies

Compare les moyennes de deux mesures sur les memes individus.

```
t.test(mesure_avant, mesure_apres, paired = TRUE)
```

bartlett.test(), **levene.test()**

Tests d'homogeneite des variances entre plusieurs groupes.

```
bartlett.test(valeur ~ groupe, data = donnees)
```

5.3 Tests non parametriques

chisq.test()

Realise un test du chi-deux d'independance entre deux variables categoriques.

```
tableau <- table(donnees$sexe, donnees$categorie)
chisq.test(tableau)
```

wilcox.test()

Alternative non parametrique au test t de Student (test de Mann-Whitney ou de Wilcoxon).

```
wilcox.test(valeur ~ groupe, data = donnees) # echantillons independants
wilcox.test(avant, apres, paired = TRUE) # echantillons apparies
```

kruskal.test()

Alternative non parametrique a l'ANOVA (test de Kruskal-Wallis).

```
kruskal.test(valeur ~ groupe, data = donnees)
```

6. Regression et correlation

6.1 Correlation

cor()

Calcule le coefficient de correlation entre deux variables.

```
cor(donnees$taille, donnees$poids) # correlation de Pearson
cor(donnees$taille, donnees$poids, method = "spearman") # correlation de rang
```

cor.test()

Teste la significativite d'une correlation.

```
cor.test(donnees$taille, donnees$poids)
```

6.2 Regression lineaire

lm()

Ajuste un modele de regression lineaire.

```
modele <- lm(poids ~ taille, data = donnees) # regression simple
modele <- lm(poids ~ taille + age, data = donnees) # regression multiple
```

summary()

Affiche un resume detaille d'un modele statistique, notamment les coefficients et leur significativite.

```
summary(modele)
```

predict()

Utilise un modele pour faire des predictions.

```
nouvelles_donnees <- data.frame(taille = c(170, 180))
predict(modele, nouvelles_donnees)
```

7. Analyse de variance (ANOVA)

7.1 ANOVA a un facteur

aov()

Realise une analyse de variance.

```
modele_anova <- aov(valeur ~ facteur, data = donnees)
summary(modele_anova)
```

TukeyHSD()

Realise des comparaisons multiples post-hoc apres une ANOVA significative.

```
TukeyHSD(modele_anova)
```

7.2 Verification des hypotheses

shapiro.test()

Teste la normalite d'une distribution (hypothese de l'ANOVA).

```
shapiro.test(residuals(modele_anova))
```

bartlett.test()

Teste l'homogeneite des variances entre groupes (hypothese de l'ANOVA).

```
bartlett.test(valeur ~ facteur, data = donnees)
```

Synthese des Fonctions R

Conclusion

Ce document presente les principales fonctions R utilisees dans les travaux pratiques de statistiques. Il est important de noter que R offre de nombreuses autres fonctions et packages specialises qui peuvent etre utilises pour des analyses plus avancees.

Pour approfondir vos connaissances, n'hesitez pas a consulter la documentation officielle de R (`help()` ou `?fonction`) ainsi que les nombreuses ressources disponibles en ligne.

Rappel des etapes typiques d'une analyse statistique avec R:

1. Importation et preparation des donnees (`read.table`, `subset`, etc.)
2. Exploration et visualisation (`summary`, `hist`, `boxplot`, etc.)
3. Analyses statistiques descriptives (`mean`, `sd`, `median`, etc.)
4. Tests d'hypotheses et modelisation (`t.test`, `lm`, `aov`, etc.)
5. Interpretation et communication des resultats